

Prompt Engineering

Mehmet Ali Akyol, PhD

Ankara Medipol University

January 2, 2026

Week 13: Emerging Trends in Prompt Engineering

The Agentic Era (2025–2026 Landscape)

- **From Chatbots to Agents:** Moving beyond reactive chatbots to proactive **Agentic AI** that plans and executes multi-step tasks autonomously.
- **Market Reality:** Enterprise AI market projected to reach \$229.3B by 2030 (McKinsey, 2024). However, Gartner predicts **40% failure rate** by 2027 due to orchestration complexity.
- **Engineering Discipline:** Prompt engineering has evolved from a “heuristic art” to a rigorous engineering discipline with formal methodologies.
- **Key Shift:** From *prompt crafting* → *system architecture* with prompts as configurable components.

The Evolution of LLM Capabilities

- **Context Windows:** From 4K tokens (GPT-3) to 2M+ tokens (Gemini 1.5, Claude 3.5)—enabling “infinite memory” workflows.
- **Reasoning Models:** OpenAI o1/o3, DeepSeek-R1, and Claude 3.5 with native chain-of-thought reasoning baked into training.
- **Tool Use Maturity:** Function calling evolved from experimental to production-grade with structured outputs (JSON mode).
- **Cost Reduction:** 99% cost reduction since 2022—enabling complex multi-agent architectures at scale.

- **Chain-of-Thought (CoT)**: Models generate intermediate reasoning steps. Improves math accuracy by **+15–40%** (Wei et al., 2022) but increases latency 2–3x.
- **Tree of Thoughts (ToT)**: Explores multiple reasoning branches, self-evaluates, and backtracks. Up to **74%** improvement on creative tasks (Yao et al., 2023).
- **Graph of Thoughts (GoT)**: Models thought processes as a DAG, enabling non-linear reasoning and merging disparate paths. **62%** quality improvement on sorting tasks (Besta et al., 2024).
- **Meta-CoT**: Dynamically selects reasoning strategy based on problem characteristics.

Reasoning Architecture Comparison

Method	Structure	Backtrack?	Best For
CoT	Linear	No	Math, Logic
Self-Consistency	Parallel	No	Ambiguous problems
ToT	Tree	Yes	Planning, Games
GoT	Graph	Yes	Complex synthesis

Trade-off: Complexity $\uparrow$ = Quality $\uparrow$ but Cost & Latency $\uparrow\uparrow$

- **Skeleton-of-Thought (SoT)**: Generates outline first, expands in parallel. Reduces latency by **2.4x** while maintaining quality (Ning et al., 2024).
- **Chain of Draft (CoD)**: Minimalistic reasoning (like expert notes). Reduces token usage by up to **92%** with minimal accuracy loss.
- **Speculative Decoding**: Draft model generates candidates; main model verifies. **2–3x** speedup without quality degradation.
- **Prompt Caching**: Reuse computed prefixes across requests. Up to **90%** cost reduction for similar queries (Anthropic, OpenAI native support).

Structured Output & Constrained Generation

- **JSON Mode:** Guaranteed valid JSON output—essential for production pipelines and tool integration.
- **Grammar-Constrained Decoding:** Force outputs to match formal grammars (CFG, regex). Tools: Outlines, Guidance, LMQL.
- **Pydantic Integration:** Define output schemas as Python classes; automatic validation and retry logic.
- **Why It Matters:** Reduces parsing failures from **15–20%** to **<1%** in production systems.

Multimodal Prompting

- **New Standard:** Multimodal models (text, image, audio, video) are the norm (Gemini 2.0, GPT-4o, Claude 3.5 Vision).
- **Core Techniques:**
 - **Region-of-Interest:** “Focus on the top-left quadrant where the chart appears.”
 - **Visual CoT:** “First describe what you see, then answer”—reduces hallucinations by **23%**.
 - **Set-of-Mark Prompting:** Overlay numbered markers on images for precise reference.
- **Emerging:** Video understanding, real-time audio agents (GPT-4o voice mode).

Retrieval-Augmented Generation (RAG) 2.0

- **Beyond Basic RAG:** Moving from simple vector search to sophisticated retrieval pipelines.
- **Advanced Patterns:**
 - **Agentic RAG:** LLM decides what/when to retrieve, reformulates queries.
 - **GraphRAG:** Combines knowledge graphs with vector retrieval for relational reasoning.
 - **Corrective RAG (CRAG):** Self-evaluates retrieval quality; falls back to web search if needed.
- **Hybrid Search:** Combining dense vectors + sparse (BM25) + reranking for **+15%** recall.

Agentic Workflows vs. Standard Chatbots

- **Core Distinction:**

- **Assistants:** Reactive, single-turn, stateless, rely on explicit user input.
- **Agents:** Proactive, multi-step, stateful, autonomous tool usage with goal persistence.

- **Cognitive Architecture:** Perception → Memory → Planning → Action → Reflection loop.

- **Multi-Agent Systems:** Specialized agents (Researcher, Critic, Executor) with emergent collaboration.

Agent Design Patterns

- **ReAct** (Yao et al., 2023): Interleaved Reasoning + Acting. Most common production pattern.
- **Plan-and-Execute**: Generate full plan upfront, then execute steps. Better for complex tasks.
- **Reflexion** (Shinn et al., 2023): Agent reflects on failures, updates strategy. Enables learning from mistakes.
- **LATS** (Language Agent Tree Search): Combines ToT with ReAct for strategic exploration.
- **Critique Pattern**: Separate “actor” and “critic” agents for self-improvement.

Orchestration Frameworks

- **LangGraph**: Stateful, cyclical workflows with persistence. Best for complex reasoning loops.
- **CrewAI**: Role-based multi-agent collaboration with built-in delegation.
- **AutoGen** (Microsoft): Conversational agents with human-in-the-loop support.
- **Semantic Kernel**: Enterprise-grade with .NET/Python, strong Azure integration.
- **Model Context Protocol (MCP)**: Anthropic's standard for agent-tool connections—"USB-C for AI."

Reliability & Production Challenges

- **Reliability Crisis:** Multi-agent systems face cascading failures; 10-step agents with 95% step accuracy = only **60%** end-to-end success.
- **Durable Execution:** **Temporal**, **Inngest** provide crash-resistant workflow engines with automatic retry.
- **Observability:** Essential tools—**LangSmith**, **Langfuse**, **Arize** for tracing agent decisions.
- **Guardrails:** **NeMo Guardrails**, **Guardrails AI** for runtime constraint enforcement.

Memory Systems for Agents

- **Short-term:** Conversation buffer within context window.
- **Long-term:** Vector databases (Pinecone, Weaviate, Chroma) for semantic retrieval.
- **Episodic:** Store and retrieve past interaction summaries for personalization.
- **Procedural:** Learned skills/tool usage patterns (emerging research).
- **Memory Consolidation:** Periodic summarization to manage growing context.

Programmatic Prompting (DSPy)

- **The Problem:** Hand-tuning prompts is brittle; model updates break carefully crafted prompts.
- **DSPy Paradigm** (Khattab et al., Stanford):
 - **Signatures:** Declarative input/output specifications.
 - **Modules:** Composable prompt patterns (ChainOfThought, ReAct).
 - **Optimizers:** MIPROv2, BootstrapFewShot auto-tune prompts.
- **Result:** +10–25% accuracy gains; prompts portable across GPT-4, Claude, Llama.

Prompt Optimization Techniques

- **Automatic Prompt Engineering (APE):** LLM generates and scores prompt candidates.
- **OPRO** (Google): Optimization by Prompting—LLM iteratively improves its own prompts.
- **Prompt Breeding:** Evolutionary algorithms mutate and crossover prompts.
- **Soft Prompts:** Learnable continuous embeddings (requires fine-tuning access).
- **Practical Tip:** Start with DSPy or manual A/B testing before complex optimization.

- **Versioning:** Prompts are code—use Git, semantic versioning, and prompt registries.
- **Evaluation-Driven Development:** Systematic testing replaces “vibes-based” checking.
- **LLM-as-Judge:** Use stronger models to evaluate weaker model outputs (GPT-4 judging GPT-3.5).
- **Platform Landscape:**
 - **Braintrust, Promptfoo:** CI/CD + evaluation pipelines.
 - **LangSmith, Langfuse:** Tracing + observability.
 - **Weights & Biases Prompts:** Experiment tracking for prompts.

Evaluation Metrics & Benchmarks

- **Task-Specific:** Accuracy, F1, BLEU, ROUGE, BERTScore.
- **LLM-as-Judge Criteria:** Relevance, coherence, faithfulness, harmlessness.
- **Agent Benchmarks:**
 - **SWE-bench:** Real GitHub issues—tests coding agents.
 - **GAIA:** General AI assistants benchmark.
 - **AgentBench:** Multi-environment agent evaluation.
- **Custom Evals:** Domain-specific test suites are essential for production.

Security Vulnerabilities (OWASP LLM Top 10)

- **LLM01: Prompt Injection:** Direct manipulation of model behavior—#1 risk.
- **LLM02: Insecure Output Handling:** Unvalidated LLM outputs executed as code/queries.
- **LLM03: Training Data Poisoning:** Malicious data influencing model behavior.
- **LLM07: Indirect Prompt Injection:** Agents reading malicious content from external sources (websites, emails, documents).
- **LLM09: Over-Reliance:** Trusting LLM outputs without verification.

Defense-in-Depth Strategies

- **Input Validation:** Sanitize, length-limit, and classify user inputs before LLM processing.
- **Prompt Scaffolding:** Rigid delimiters, role separation (<user>, <system>).
- **Output Filtering:** Constitutional classifiers, content moderation APIs.
- **Least Privilege:** Agents should have minimal permissions; no direct DB/shell access.
- **Human-in-the-Loop:** Require approval for high-risk actions (payments, deletions).

Emerging Security Considerations

- **Data Privacy:** PII leakage in prompts, training data extraction attacks.
- **Model Extraction:** Adversarial queries to reverse-engineer proprietary models.
- **Supply Chain:** Third-party prompts, plugins, and MCP servers as attack vectors.
- **Regulatory Compliance:** EU AI Act, NIST AI RMF require documented risk management.
- **Red Teaming:** Proactive adversarial testing is becoming standard practice.

Strategic Recommendations for 2025–2026

- **Think Systems, Not Prompts:** Move from single prompts to modular, testable architectures.
- **Embrace Compound AI:** Combine multiple models, retrievers, and tools for robust solutions.
- **Invest in Evaluation:** Build comprehensive eval suites before scaling.
- **Optimize Strategically:** Balance quality, latency, and cost for your use case.
- **Security by Design:** Assume adversarial inputs; implement defense-in-depth.

The Prompt Engineer's Evolving Role

- **2023:** Prompt Crafter—writing clever prompts for single tasks.
- **2024:** Prompt Engineer—systematic testing, versioning, optimization.
- **2025:** AI Systems Architect—designing multi-agent workflows, RAG pipelines.
- **2026+:** AI Product Engineer—end-to-end ownership of AI-native products.
- **Key Skills:** Systems thinking, evaluation design, security awareness, domain expertise.

What to Watch: 2026 Predictions

- **Reasoning Models:** Native CoT in base models (o1-style) becomes standard.
- **Agent Frameworks:** Consolidation around 2–3 dominant orchestration platforms.
- **Prompt Commoditization:** Basic prompting becomes automated; focus shifts to architecture.
- **Multimodal Agents:** Video understanding + real-time audio enables new use cases.
- **Regulation:** AI governance frameworks impact deployment practices.

Key Takeaways

- **Paradigm Shift:** From prompt crafting to AI systems architecture.
- **Compound AI:** Future systems combine multiple models, tools, and retrieval.
- **Reliability Engineering:** Evaluation, observability, and guardrails are essential.
- **Security Imperative:** Adversarial thinking must be embedded in design.
- **Continuous Learning:** The field evolves monthly—stay curious and adaptive.

Recommended Resources

- **Papers:** “DSPy” (Khattab et al.), “ReAct” (Yao et al.), “Tree of Thoughts” (Yao et al.)
- **Courses:** DeepLearning.AI “Building Agentic AI”, Anthropic Prompt Engineering Guide
- **Newsletters:** The Batch (Andrew Ng), Simon Willison’s Blog, Latent Space Podcast
- **Communities:** r/LocalLLaMA, AI Discord servers, LangChain/LlamaIndex communities
- **Benchmarks:** SWE-bench, GAIA, MMLU, HumanEval for tracking progress

Thank You!

Dr. Mehmet Ali Akyol

mehmetali.akyol@gmail.com

Questions?